



DEPARTMENT OF INFORMATION & COMMUNICATION TECHNOLOGY
SESSION II 2022/2023

DFP40203
PYTHON PROGRAMMING

CONTINUOUS ASSESSMENT:
PROBLEM BASED TASK 2

NAME	MATRIX NUMBER
MUHAMMAD REDZUAN BIN PARSIKUN	03DDT21F1005
CHAIRIS A/L PUM	03DDT21F1088
SYAKIR ZUFAYRIN BIN KHAIRUL FAIZAL	03DDT21F1085

CLASS : DDT4A

LECTURER :
SITI NUR THAZLIAH BINTI MOHD THAZALI

Introduction :

It demonstrates a basic restaurant order system. The GUI displays a menu with different dishes and allows the user to add or remove items from their order.

The code defines a dictionary called prices that stores the prices of various dishes in the restaurant. It then creates a Tkinter window as the main interface.

The program includes functions such as add() and remove() that are triggered when the user interacts with the GUI buttons. These functions update the order details and calculate the total cost of the order based on the selected items and saved it in file or receipt

Overall, this code provides a foundation for a restaurant order system GUI, allowing users to select dishes and manage their order effectively.

a) Coding :

```
1  import tkinter as tk
2  from tkinter import ttk
3  from tkinter import*
4  from PIL import Image, ImageTk
5  import random
6  from datetime import date
7  from datetime import datetime
8
9  prices = {
10     "Fried Calamari" : 10,
11     "Beach Burger" : 14,
12     "Salmon Wonder" : 23,
13     "Shrimp Tacos" : 15,
14     "Sushi Platter" : 25,
15     "Empanadas" : 10,
16 }
17
18 root = Tk()
19
20 root.title("TTC - Binary Restaurant")
21
22 # -----FUNCTIONS----- #
23
24 #region Generating a random Order ID when starting a new order
25 def ORDER_ID():
```

```

25 def ORDER_ID():
26     numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9] # List of numbers
27     letters = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U',
28               'V', 'W', 'X', 'Y', 'Z'] # List of letters
29     order_id = "BIN_" # Initial order ID string
30     random_letters = "" # Empty string to store random letters
31     random_digits = "" # Empty string to store random digits
32     for i in range(0,3): # Loop three times
33         random_letters += random.choice(letters) # Append a random letter to random_letters
34         random_digits += str(random.choice(numbers)) # Append a random digit to random_digits
35
36     order_id += random_letters + random_digits # Concatenate order_id with random_letters and random_digits
37     return order_id # Return the generated order ID
38 #endregion
39
40 #region Add to Order Button

```

```

40 #region Add to Order Button
41 def add():
42     # Updating the transaction label
43     current_order = orderTransaction.cget("text") # Get the current text of the transaction label
44     added_dish = displayLabel.cget("text") + "...." + str(prices[displayLabel.cget("text")]) + "$ " # Create the text for the added dish
45     updated_order = current_order + added_dish # Concatenate the current order and the added dish
46     orderTransaction.configure(text=updated_order) # Update the text of the transaction label
47
48     # Updating the order total label
49     order_total = orderTotalLabel.cget("text").replace("TOTAL : ", "") # Get the current order total
50     order_total = order_total.replace("$", "") # Remove the dollar sign from the order total
51     updated_total = int(order_total) + prices[displayLabel.cget("text")] # Calculate the updated order total
52     orderTotalLabel.configure(text="TOTAL : " + str(updated_total) + "$") # Update the text of the order total label
53 #endregion
54
55 #region Remove Button Function

```

```

56 def remove():
57     dish_to_remove = displayLabel.cget("text") + "..." + str(prices[displayLabel.cget("text")]) # Get the text of the dish to remove
58     transaction_list = orderTransaction.cget("text").split("$ ") # Split the transaction label text into a list
59     transaction_list.pop(len(transaction_list) - 1) # Remove the last empty item from the list
60
61     if dish_to_remove in transaction_list: # Check if the dish to remove is in the transaction list
62         # Update transaction label
63         transaction_list.remove(dish_to_remove) # Remove the dish to remove from the transaction list
64         updated_order = "" # Empty string to store the updated order
65         for item in transaction_list: # Iterate over the items in the transaction list
66             updated_order += item + "$ " # Append each item to the updated order with a dollar sign
67
68         orderTransaction.configure(text=updated_order) # Update the text of the transaction label
69
70         # Update transaction total
71         order_total = orderTotalLabel.cget("text").replace("TOTAL : ", "") # Get the current order total
72         order_total = order_total.replace("$", "") # Remove the dollar sign from the order total
73         updated_total = int(order_total) - prices[displayLabel.cget("text")] # Calculate the updated order total
74         orderTotalLabel.configure(text="TOTAL : " + str(updated_total) + "$") # Update the text of the order total label
75
76
77 #endregion

```

```

79 #region Display Button Functions
80 def displayCalamari():
81     # Set the style of the calamari dish frame
82     calamariDishFrame.configure(
83         relief="sunken",
84         style="SelectedDish.TFrame"
85     )
86     # Set the style of other dish frames to the default style
87     salmonDishFrame.configure(style="DishFrame.TFrame")
88     empanadasDishFrame.configure(style="DishFrame.TFrame")
89     sushiDishFrame.configure(style="DishFrame.TFrame")
90     shrimpDishFrame.configure(style="DishFrame.TFrame")
91     burgerDishFrame.configure(style="DishFrame.TFrame")
92
93     # Configure the display label for calamari dish
94     displayLabel.configure(
95         image=calamariImage, # Set the image for calamari dish
96         text="Fried Calamari", # Set the text for calamari dish
97         font=('Helvetica', 14, "bold"), # Set the font for the text
98         foreground="white", # Set the text color
99         compound="bottom", # Set the image and text arrangement
100         padding=(5, 5, 5, 5), # Set the padding around the label
101     )

```

```

103 def displayBurger():
104     # Set the style of the burger dish frame
105     burgerDishFrame.configure(
106         relief="sunken",
107         style="SelectedDish.TFrame"
108     )
109     # Set the style of other dish frames to the default style
110     salmonDishFrame.configure(style="DishFrame.TFrame")
111     empanadasDishFrame.configure(style="DishFrame.TFrame")
112     sushiDishFrame.configure(style="DishFrame.TFrame")
113     shrimpDishFrame.configure(style="DishFrame.TFrame")
114     calamariDishFrame.configure(style="DishFrame.TFrame")
115     # Configure the display label for the burger dish
116     displayLabel.configure(
117         text="Beach Burger", # Set the text for the burger dish
118         font=('Helvetica', 14, "bold"), # Set the font for the text
119         foreground="white", # Set the text color
120         image=burgerImage, # Set the image for the burger dish
121         compound="bottom", # Set the image and text arrangement
122         padding=(5, 5, 5, 5), # Set the padding around the label
123     )
124
125

```

```

126 def displaySalmon():
127     # Set the style of the salmon dish frame
128     salmonDishFrame.configure(
129         relief="sunken",
130         style="SelectedDish.TFrame"
131     )
132     # Set the style of other dish frames to the default style
133     calamariDishFrame.configure(style="DishFrame.TFrame")
134     empanadasDishFrame.configure(style="DishFrame.TFrame")
135     sushiDishFrame.configure(style="DishFrame.TFrame")
136     shrimpDishFrame.configure(style="DishFrame.TFrame")
137     burgerDishFrame.configure(style="DishFrame.TFrame")
138     # Configure the display label for the salmon dish
139     displayLabel.configure(
140         text="Salmon Wonder", # Set the text for the salmon dish
141         font=('Helvetica', 14, "bold"), # Set the font for the text
142         foreground="white", # Set the text color
143         image=salmonImage, # Set the image for the salmon dish
144         compound="bottom", # Set the image and text arrangement
145         padding=(5, 5, 5, 5), # Set the padding around the label
146     )
147

```

```

148 def displayTacos():
149     # Set the style of the shrimp dish frame
150     shrimpDishFrame.configure(
151         relief="sunken",
152         style="SelectedDish.TFrame"
153     )
154     # Set the style of other dish frames to the default style
155     salmonDishFrame.configure(style="DishFrame.TFrame")
156     empanadasDishFrame.configure(style="DishFrame.TFrame")
157     sushiDishFrame.configure(style="DishFrame.TFrame")
158     calamariDishFrame.configure(style="DishFrame.TFrame")
159     burgerDishFrame.configure(style="DishFrame.TFrame")
160     # Configure the display label for the shrimp dish
161     displayLabel.configure(
162         text="Shrimp Tacos", # Set the text for the shrimp dish
163         font=('Helvetica', 14, "bold"), # Set the font for the text
164         foreground="white", # Set the text color
165         image=shrimpImage, # Set the image for the shrimp dish
166         compound="bottom", # Set the image and text arrangement
167         padding=(5, 5, 5, 5), # Set the padding around the label
168     )
169
170 def displayEmpanadas():
171     # Set the style of the empanadas dish frame
172     empanadasDishFrame.configure(
173         relief="sunken",
174         style="SelectedDish.TFrame"
175     )
176     # Set the style of other dish frames to the default style
177     salmonDishFrame.configure(style="DishFrame.TFrame")
178     calamariDishFrame.configure(style="DishFrame.TFrame")
179     sushiDishFrame.configure(style="DishFrame.TFrame")
180     shrimpDishFrame.configure(style="DishFrame.TFrame")
181     burgerDishFrame.configure(style="DishFrame.TFrame")
182     # Configure the display label for the empanadas dish
183     displayLabel.configure(
184         text="Empanadas", # Set the text for the empanadas dish
185         font=('Helvetica', 14, "bold"), # Set the font for the text
186         foreground="white", # Set the text color
187         image=empanadasImage, # Set the image for the empanadas dish
188         compound="bottom", # Set the image and text arrangement
189         padding=(5, 5, 5, 5), # Set the padding around the label
190     )
191

```

```

192     def displaySushi():
193         # Set the style of the sushi dish frame
194         sushiDishFrame.configure(
195             relief="sunken",
196             style="SelectedDish.TFrame"
197         )
198         # Set the style of other dish frames to the default style
199         salmonDishFrame.configure(style="DishFrame.TFrame")
200         empanadasDishFrame.configure(style="DishFrame.TFrame")
201         calamariDishFrame.configure(style="DishFrame.TFrame")
202         shrimpDishFrame.configure(style="DishFrame.TFrame")
203         burgerDishFrame.configure(style="DishFrame.TFrame")
204         # Configure the display label for the sushi dish
205         displayLabel.configure(
206             image=sushiImage, # Set the image for the sushi dish
207             text="Sushi Platter", # Set the text for the sushi dish
208             font=('Helvetica', 14, "bold"), # Set the font for the text
209             foreground="white", # Set the text color
210             compound="bottom", # Set the image and text arrangement
211             padding=(5, 5, 5, 5), # Set the padding around the label
212         )
213     #endregion
214

```

```

215 #region Generating Receipt from Order Button
216 def order():
217     new_receipt = orderIDLabel.cget("text") # Get the text of the order ID label
218     new_receipt = new_receipt.replace("ORDER ID : ", "") # Remove the "ORDER ID: " prefix from the order ID
219     transaction_list = orderTransaction.cget("text").split("$ ") # Split the transaction label text into a list
220     transaction_list.pop(len(transaction_list) - 1) # Remove the last empty item from the list
221
222     order_day = date.today() # Get the current date
223     order_time = datetime.now() # Get the current time
224
225     for item in transaction_list: # Iterate over the items in the transaction list
226         item += "$ " # Append a dollar sign to each item
227
228     with open(new_receipt, 'w') as file: # Open the receipt file in write mode
229         file.write("The Binary") # Write the restaurant name
230         file.write("\n")
231         file.write("_____") # Write a separator
232         file.write("\n")

```



```

233     file.write(order_day.strftime("%x")) # Write the order date
234     file.write("\n")
235     file.write(order_time.strftime("%X")) # Write the order time
236     file.write("\n\n")
237     for item in transaction_list: # Iterate over the items in the transaction list
238         file.write(item + "\n") # Write each item to the file
239     file.write("\n\n")
240     file.write(orderTotalLabel.cget("text")) # Write the order total
241
242     orderTotalLabel.configure(text="TOTAL : 0$") # Reset the order total label
243     orderIDLabel.configure(text="ORDER ID: " + ORDER_ID()) # Generate a new order ID
244     orderTransaction.configure(text="") # Clear the transaction label
245
246 #endregion
247
248 # ----- STYLING AND IMAGES ----- #
249
250 #region Style configurations
251 s = ttk.Style() # Set the style for different widgets using ttk.Style()
252 # Configure the styles for different frames
253 s.configure('MainFrame.TFrame', background="#2B2B28") # Styling for the main frame
254 s.configure('MenuFrame.TFrame', background="#4A4A48") # Styling for the menu frame
255 s.configure('DisplayFrame.TFrame', background="#0F1110") # Styling for the display frame
256 s.configure('OrderFrame.TFrame', background="#B7C4CF") # Styling for the order frame
257 s.configure('DishFrame.TFrame', background="#4A4A48", relief="raised") # Styling for the dish frame
258 s.configure('SelectedDish.TFrame', background="#C4DFAA") # Styling for the selected dish frame
259 # Configure the styles for different labels
260 s.configure('MenuLabel.TLabel',
261             background="#0F1110",
262             font=("Arial", 13, "italic"),
263             foreground="white",
264             padding=(5, 5, 5, 5),
265             width=21
266             ) # Styling for the menu label
267 s.configure('orderTotalLabel.TLabel',
268             background="#0F1110",
269             font=("Arial", 10, "bold"),
270             foreground="white",
271             padding=(2, 2, 2, 2),
272             anchor="w"
273             ) # Styling for the order total label

```



```

274 s.configure('orderTransaction.TLabel',
275             background="#444444",
276             font=('Helvetica', 12),
277             foreground="white",
278             wraplength=170,
279             anchor="nw",
280             padding=(3, 3, 3, 3)
281             ) # Styling for the order transaction label
282
283
284 # endregion
285
286 # region Images
287 # Top Banner images
288 LogoImageObject = Image.open("Images/Binary Logo.png").resize((130, 130)) # Load and resize the logo image
289 LogoImage = ImageTk.PhotoImage(LogoImageObject) # Create a PhotoImage object from the logo image
290
291 TopBannerImageObject = Image.open("Images/restaurant top banner.jpg").resize((800, 130)) # Load and resize the top banner image
292 TopBannerImage = ImageTk.PhotoImage(TopBannerImageObject) # Create a PhotoImage object from the top banner image
293
294 displayDefaultImageObject = Image.open("Images/display - Default.png").resize((350, 360)) # Load and resize the default display image
295 displayDefaultImage = ImageTk.PhotoImage(displayDefaultImageObject) # Create a PhotoImage object from the default display image
296
297 calamariImageObject = Image.open("Images/menu/fried calamari.png").resize((350, 334)) # Load and resize the calamari dish image
298 calamariImage = ImageTk.PhotoImage(calamariImageObject) # Create a PhotoImage object from the calamari dish image
299
300 burgerImageObject = Image.open("Images/menu/beach burger.png").resize((350, 334)) # Load and resize the burger dish image
301 burgerImage = ImageTk.PhotoImage(burgerImageObject) # Create a PhotoImage object from the burger dish image
302
303 salmonImageObject = Image.open("Images/menu/salmon wild rice.png").resize((350, 334)) # Load and resize the salmon dish image
304 salmonImage = ImageTk.PhotoImage(salmonImageObject) # Create a PhotoImage object from the salmon dish image
305
306 shrimpImageObject = Image.open("Images/menu/shrimp tacos.png").resize((350, 334))
307 shrimpImage = ImageTk.PhotoImage(shrimpImageObject)
308
309 sushiImageObject = Image.open("Images/menu/sushi platter.png").resize((350, 334)) # Load and resize the sushi dish image
310 sushiImage = ImageTk.PhotoImage(sushiImageObject) # Create a PhotoImage object from the sushi dish image
311
312 empanadasImageObject = Image.open("Images/menu/empanadas.png").resize((350, 334)) # Load and resize the empanadas dish image
313 empanadasImage = ImageTk.PhotoImage(empanadasImageObject) # Create a PhotoImage object from the empanadas dish image
314
315
316 #endregion

```

```

318 #----- WIDGETS ----- #
319
320 # region Frames
321
322 # Section Frames
323 mainFrame = ttk.Frame(root, width = 800, height = 580, style = 'MainFrame.TFrame') # Main frame of the application
324 mainFrame.grid(row = 0, column = 0, sticky = "NSEW")
325
326 topBannerFrame = ttk.Frame(mainFrame) # Frame for the top banner
327 topBannerFrame.grid(row = 0, column = 0, sticky = "NSEW", columnspan = 3)
328
329 menuFrame = ttk.Frame(mainFrame, style = 'MenuFrame.TFrame') # Frame for the menu section
330 menuFrame.grid(row = 1, column = 0, padx = 3, pady = 3, sticky = "NSEW")
331
332 displayFrame = ttk.Frame(mainFrame, style = "DisplayFrame.TFrame") # Frame for the display section
333 displayFrame.grid(row = 1, column = 1, padx = 3, pady = 3, sticky = "NSEW")
334
335 orderFrame = ttk.Frame(mainFrame, style = "OrderFrame.TFrame") # Frame for the order section
336 orderFrame.grid(row = 1, column = 2, padx = 3, pady = 3, sticky = "NSEW")
337
338 # Dish Frames
339 calamariDishFrame = ttk.Frame(menuFrame, style = "DishFrame.TFrame") # Frame for the calamari dish
340 calamariDishFrame.grid(row = 1, column = 0, sticky = "NSEW")
341
342 burgerDishFrame = ttk.Frame(menuFrame, style = "DishFrame.TFrame") # Frame for the burger dish
343 burgerDishFrame.grid(row = 2, column = 0, sticky = "NSEW")
344
345 salmonDishFrame = ttk.Frame(menuFrame, style = "DishFrame.TFrame") # Frame for the salmon dish
346 salmonDishFrame.grid(row = 3, column = 0, sticky = "NSEW")
347
348 shrimpDishFrame = ttk.Frame(menuFrame, style = "DishFrame.TFrame") # Frame for the shrimp dish
349 shrimpDishFrame.grid(row = 4, column = 0, sticky = "NSEW")
350
351 sushiDishFrame = ttk.Frame(menuFrame, style = "DishFrame.TFrame") # Frame for the sushi dish
352 sushiDishFrame.grid(row = 5, column = 0, sticky = "NSEW")
353
354 empanadasDishFrame = ttk.Frame(menuFrame, style = "DishFrame.TFrame") # Frame for the empanadas dish
355 empanadasDishFrame.grid(row = 6, column = 0, sticky = "NSEW")
356
357 #endregion
358

```

```

359 # region Top Banner Section
360
361 LogoLabel = ttk.Label(topBannerFrame, image = LogoImage, background = "#0F1110") # Label for the logo image
362 LogoLabel.grid(row = 0, column = 0, sticky = "W")
363
364 RestaurantBannerLabel = ttk.Label(topBannerFrame, image = TopBannerImage, background = "#0F1110")
365 # Label for the restaurant banner image
366 RestaurantBannerLabel.grid(row = 0, column = 1, sticky = "NSEW")
367
368 # endregion
369
370 # region Menu Section
371 MainMenuLabel = ttk.Label(menuFrame, text = "MENU", style = "MenuLabel.TLabel") # Label for the menu title
372 MainMenuLabel.grid(row = 0, column = 0, sticky = "WE")
373 MainMenuLabel.configure(
374     anchor = "center",
375     font = ("Helvetica", 14, "bold")
376 )
377
378 CalamariDishLabel = ttk.Label(calamariDishFrame, text = "Fried Calamari ..... 10$", style = "MenuLabel.TLabel") # Label
379 CalamariDishLabel.grid(row = 0, column = 0, padx = 10, pady = 10, sticky = "W")
380
381
382 BurgerDishLabel = ttk.Label(burgerDishFrame, text = "Beach Burger ..... 14$", style = "MenuLabel.TLabel") # Label for the burger dish
383 BurgerDishLabel.grid(row = 0, column = 0, padx = 10, pady = 10, sticky = "W")
384
385 SalmonDishLabel = ttk.Label(salmonDishFrame, text = "Salmon Wonder ..... 23$", style = "MenuLabel.TLabel") # Label for the salmon dish
386 SalmonDishLabel.grid(row = 0, column = 0, padx = 10, pady = 10, sticky = "W")
387
388 ShrimpDishLabel = ttk.Label(shrimpDishFrame, text = "Shrimp Tacos ..... 15$", style = "MenuLabel.TLabel") # Label for the shrimp dish
389 ShrimpDishLabel.grid(row = 0, column = 0, padx = 10, pady = 10, sticky = "W")
390
391 SushiDishLabel = ttk.Label(sushiDishFrame, text = "Sushi Platter ..... 25$", style = "MenuLabel.TLabel") # Label for the sushi dish
392 SushiDishLabel.grid(row = 0, column = 0, padx = 10, pady = 10, sticky = "W")
393
394 EmpanadasDishLabel = ttk.Label(empanadasDishFrame, text = "Empanadas .... 10$", style = "MenuLabel.TLabel") # Label for the empanadas dish
395 EmpanadasDishLabel.grid(row = 0, column = 0, padx = 10, pady = 10, sticky = "W")

```

```

396 # Buttons
397 CalamariDisplayButton = ttk.Button(calamariDishFrame, text = "Display", command = displayCalamari) # Button to display calamari dish
398 CalamariDisplayButton.grid(row = 0, column = 1, padx = 10)
399
400 BurgerDisplayButton = ttk.Button(burgerDishFrame, text = "Display", command = displayBurger) # Button to display burger dish
401 BurgerDisplayButton.grid(row = 0, column = 1, padx = 10)
402
403 SalmonDisplayButton = ttk.Button(salmonDishFrame, text = "Display", command = displaySalmon) # Button to display salmon dish
404 SalmonDisplayButton.grid(row = 0, column = 1, padx = 10)
405
406 ShrimpDisplayButton = ttk.Button(shrimpDishFrame, text = "Display", command = displayTacos) # Button to display shrimp dish
407 ShrimpDisplayButton.grid(row = 0, column = 1, padx = 10)
408
409 SushiDisplayButton = ttk.Button(sushiDishFrame, text = "Display", command = displaySushi) # Button to display sushi dish
410 SushiDisplayButton.grid(row = 0, column = 1, padx = 10)
411
412 EmpanadasDisplayButton = ttk.Button(empanadasDishFrame, text = "Display", command = displayEmpanadas) # Button to display empanadas dish
413 EmpanadasDisplayButton.grid(row = 0, column = 1, padx = 10)
414
415
416
417 # region Order Section
418 orderTitleLabel = ttk.Label(orderFrame, text = "ORDER") # Label for the order title
419 orderTitleLabel.configure(
420     foreground="white", background="black",
421     font=("Helvetica", 14, "bold"), anchor = "center",
422     padding = (5, 5, 5, 5),
423 )
424 orderTitleLabel.grid(row = 0, column = 0, sticky = "EW")
425
426 orderIDLabel = ttk.Label(orderFrame, text = "ORDER ID : " + ORDER_ID()) # Label for the order ID
427 orderIDLabel.configure(
428     background = "black",
429     foreground = "white",
430     font = ("Helvetica", 11, "italic"),
431     anchor = "center",
432 )
433 orderIDLabel.grid(row = 1, column = 0, sticky = "EW", pady = 1)
434
435 orderTransaction = ttk.Label(orderFrame, style = 'orderTransaction.TLabel') # Label for the order transaction
436 orderTransaction.grid(row = 2, column = 0, sticky = "NSEW")
437
438 orderTotalLabel = ttk.Label(orderFrame, text = "TOTAL : 0$", style = "orderTotalLabel.TLabel") # Label for the order total
439 orderTotalLabel.grid(row = 3, column = 0, sticky = "EW")
440
441 orderButton = ttk.Button(orderFrame, text = "ORDER", command = order) # Button to place the order
442 orderButton.grid(row = 4, column = 0, sticky = "EW")
443

```

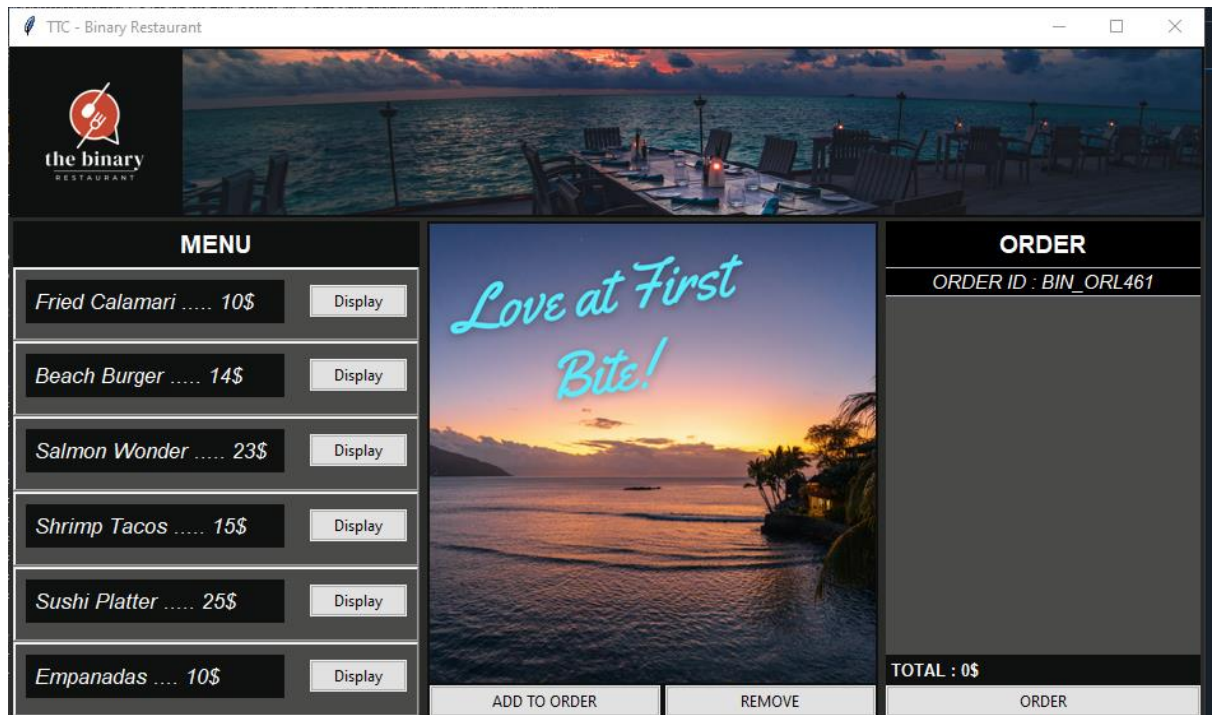
```

447 # region Display Section
448 displayLabel = ttk.Label(displayFrame, image = displayDefaultImage) # Label for the display image
449 displayLabel.grid(row = 0, column = 0, sticky = "NSEW", columnspan = 2)
450 displayLabel.configure(background = "#0F1110")
451
452 addOrderButton = ttk.Button(displayFrame, text = "ADD TO ORDER", command = add) # Button to add item to the order
453 addOrderButton.grid(row = 1, column = 0, padx = 2, sticky = "NSEW")
454
455 removeOrderButton = ttk.Button(displayFrame, text = "REMOVE", command = remove) # Button to remove item from the order
456 removeOrderButton.grid(row = 1, column = 1, padx = 2, sticky = "NSEW")
457
458 # endregion
459
460 #----- GRID CONFIGURATIONS -----#
461
462 # Configuring the grid layout for the mainFrame
463 mainFrame.columnconfigure(2, weight = 1) # The 3rd column expands horizontally when the window is resized
464 mainFrame.rowconfigure(1, weight = 1) # The 2nd row expands vertically when the window is resized
465
466 # Configuring the grid layout for the menuFrame
467 menuFrame.columnconfigure(0, weight = 1) # The 1st column expands horizontally when the window is resized
468 menuFrame.rowconfigure(1, weight = 1) # The 2nd row expands vertically when the window is resized
469 menuFrame.rowconfigure(2, weight = 1) # The 3rd row expands vertically when the window is resized
470 menuFrame.rowconfigure(3, weight = 1) # The 4th row expands vertically when the window is resized
471 menuFrame.rowconfigure(4, weight = 1) # The 5th row expands vertically when the window is resized
472 menuFrame.rowconfigure(5, weight = 1) # The 6th row expands vertically when the window is resized
473 menuFrame.rowconfigure(6, weight = 1) # The 7th row expands vertically when the window is resized
474
475 # Configuring the grid layout for the orderFrame
476 orderFrame.columnconfigure(0, weight = 1) # The 1st column expands horizontally when the window is resized
477 orderFrame.rowconfigure(2, weight = 1) # The 3rd row expands vertically when the window is resized
478
479
480
481
482 root.mainloop() # The 'mainloop' function is called to start the main event loop of the Tkinter application.

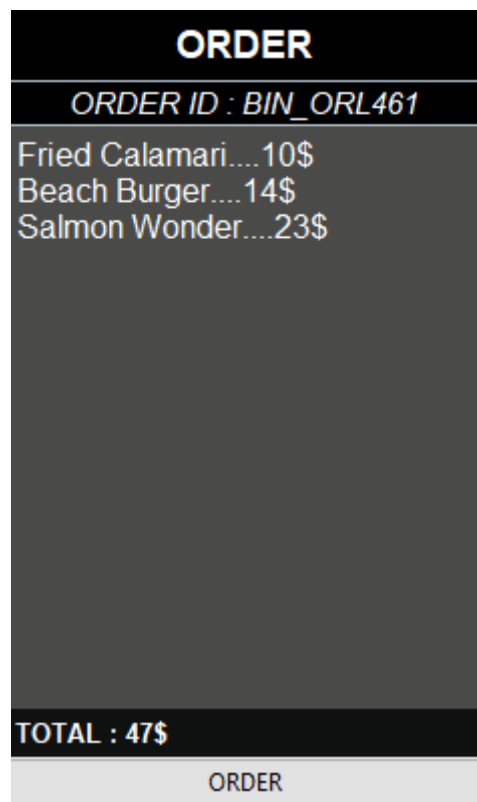
```

b) Output :

Interface output:



Order Output:



File directory:

 Images	20/5/2023 8:35 PM	File folder	
 BIN_ORL461	20/5/2023 10:06 PM	File	1 KB
 BIN_VNU722	20/5/2023 8:40 PM	File	1 KB
 main.py	20/5/2023 10:00 PM	Python File	24 KB

```
BIN_ORL461 - Notepad
File Edit Format View Help
The Binary
05/20/23
22:06:09

Fried Calamari....10
Beach Burger....14
Salmon Wonder....23

TOTAL : 47$
```


c) Conclusion:

It demonstrates a basic restaurant order system. The GUI displays a menu with different dishes and allows the user to add or remove items from their order.

The code defines a dictionary called prices that stores the prices of various dishes in the restaurant. It then creates a Tkinter window as the main interface.

The program includes functions such as add() and remove() that are triggered when the user interacts with the GUI buttons. These functions update the order details and calculate the total cost of the order based on the selected items and saved it in file or receipt

Overall, this code provides a foundation for a restaurant order system GUI, allowing users to select dishes and manage their order effectively.